习题课-第15周

北京大學

目录

- 第五次书面作业
- 第十到十二次上机作业
- 补充内容

判断题

- 1. 对任意一个连通的、无环的无向图，从图中移除任意一条边得到的图均不连通

判断题

- 1. 对任意一个连通的、无环的无向图，从图中移除任意一条边得到的图均不连通
  - 正确。这样的无向图构成生成树，每条边都是维持整个图连通所必须的

判断题

- 1. 对任意一个连通的、无环的无向图，从图中移除任意一条边得到的图均不连通
  - 正确。这样的无向图构成生成树，每条边都是维持整个图连通所必须的
- 2. 对于包含负权边，但不包含负权回路的有向图，迪杰斯特拉（Dijkstra）算法能够在其上正确求解单源最短路径问题。

判断题

- 1. 对任意一个连通的、无环的无向图，从图中移除任意一条边得到的图均不连通
  - 正确。这样的无向图构成生成树，每条边都是维持整个图连通所必须的
- 2. 对于包含负权边，但不包含负权回路的有向图，迪杰斯特拉（Dijkstra）算法能够在其上正确求解单源最短路径问题。
  - 错误。Dijkstra 不适用于包含负权回路的图。其正确性证明中假设了所有边权非负。

判断题

- 3. 为了判断有向图中是否有回路，可以利用拓扑排序算法，也可以使用广度优先遍历，或者最短路径算法

判断题

- 3. 为了判断有向图中是否有回路，可以利用拓扑排序算法，也可以使用广度优先遍历，或者最短路径算法
  - 错误。
  - 拓扑排序算法可以用于检测回路，如果运行途中不存在入度为 0 的结点，就说明存在回路。
  - 一般的 BFS 算法无法检测回路，Floyd 算法可以用于检测负回路，但无法检测所有的一般回路。

判断题

- 3. 为了判断有向图中是否有回路，可以利用拓扑排序算法，也可以使用广度优先遍历，或者最短路径算法
  - 错误。
  - 拓扑排序算法可以用于检测回路，如果运行途中不存在入度为 0 的结点，就说明存在回路。
  - 一般的 BFS 算法无法检测回路，Floyd 算法可以用于检测负回路，但无法检测所有的一般回路。
- 4. 如果 G’ 是无向图 G 的生成树，则 G’ 是图 G 的一个极小连通子图、一个无环子图、一个连通分量。

判断题

- 3. 为了判断有向图中是否有回路，可以利用拓扑排序算法，也可以使用广度优先遍历，或者最短路径算法
  - 错误。
  - 拓扑排序算法可以用于检测回路，如果运行途中不存在入度为 0 的结点，就说明存在回路。
  - 一般的 BFS 算法无法检测回路，Floyd 算法可以用于检测负回路，但无法检测所有的一般回路。
- 4. 如果 G’ 是无向图 G 的生成树，则 G’ 是图 G 的一个极小连通子图、一个无环子图、一个连通分量。
  - 错误。生成树是极小的连通子图，连通分量是极大的连通子图。

判断题

- 什么是极大 / 极小连通子图？
  - 直观理解：无法再加入结点和边 / 删除结点和边的连通子图
- 形式化的定义：
  - 定义图的包含关系（偏序关系）：$G_1 \leq G_2$ 如果 G_1 是 G_2 的子图
    - 即，$V_1 \subseteq V_2$，且 $E_1 \subseteq E_2$
  - 对于一个无向图 G，我们考虑其所有的连通子图构成的集合 S
  - S 上同样可以定义该包含关系
  - 如果 $G_1 \in S$，且不存在图 $G_2 \in S$，使得 G_1 是 G_2 的子图，即 $G_1 \leq G_2$，则称 G_1 是 G 的极大连通子图
  - 类似地，可以定义极小连通子图

判断题

- 5. 若带权连通无向图的边的权值互不相同，则其具有唯一的最小生成树。

判断题

- 5. 若带权连通无向图的边的权值互不相同，则其具有唯一的最小生成树。
  - 正确。
  - 按照 Prim 或 Kruskal 算法构建生成树时，生成树不唯一的情况仅发生在，有两条权值相等的边均为当前一步可选择的最小权值的边。
  - 实际上最小生成树具有如下性质：同一个图的任意两棵最小生成树，将它们的边分别按权值排序，得到的权值序列是相同的

判断题

- 性质：同一个图的任意两棵最小生成树，将它们的边分别按权值排序，得到的权值序列是相同的
  - 记按权值排序的边的序列分别为 $A = a_1, a_2, \cdots, a_n$；$B = b_1, b_2, \cdots, b_n$
  - 考虑第一个不同的边 $a_i \neq b_i$，不妨假设 $W(a_i) \geq W(b_i)$
  - (1) b_i 也出现在 A 中：
    - 由于 a_i 之前的边都与 B 相同，b_i 一定出现在 a_i 之后，即 $W(a_i) \leq W(b_i)$
    - 因此实际上 $W(a_i) \geq W(b_i)$，二者权值相等。
    - 即两条权值相等的边在排序后的序列中位置不同。
    - 在 A 中调换二者，就可以使 $a_i = b_i$，继续寻找下一个不同的位置

判断题

- 性质：同一个图的任意两棵最小生成树，将它们的边分别按权值排序，得到的权值序列是相同的
  - 记按权值排序的边的序列分别为 $A = a_1, a_2, \cdots, a_n$；$B = b_1, b_2, \cdots, b_n$
  - 考虑第一个不同的边 $a_i \neq b_i$，不妨假设 $W(a_i) \geq W(b_i)$
  - (2) b_i 不出现在 A 中：
    - 将 b_i 加入 A 中形成回路 C 。
    - b_i 一定是 C 中权重最大的边，否则就可以通过替换使 A 的权重减小
    - C 中至少有一条边 $a_j \notin B$，否则意味着 B 中包含回路
    - 又因为 b_i 之前的边都与 A 相同，因此 a_j 在 a_i 之后，$W(a_j) \geq W(a_i) \geq W(b_i)$
    - 因此实际上 $W(a_j) = W(b_i)$，即 A, B 包含权值相等的不同边
    - 在 C 中用 b_i 替换 a_j，保持了 A 的权值序列不变，但转化成了情况 (1)

判断题

- 6. 对于任意有向图，通过 DFS 或者 BFS 都可以计算出其拓扑排序。
  - 错误。只有有向无环图 (DAG) 才具有拓扑排序
  - 对于 DAG，基于 DFS、BFS 都可以设计算法来计算拓扑排序
- 基于 BFS 的拓扑排序算法：
  - 首先将入度为 0 的顶点加入队列
  - 不断取出队头的元素，加入拓扑排序序列，删除出边，并更新邻居的入度，判断并将邻居加入队列
  - 这一过程可以看作是基于“拓扑层级”的 BFS

判断题

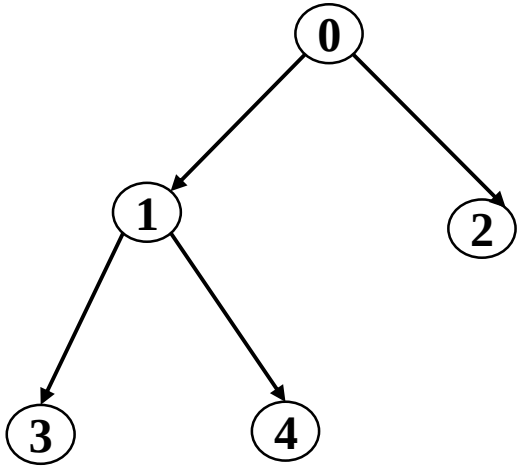
- 基于 DFS 的拓扑排序算法：
  - 相比于基本的 DFS 算法，用于拓扑排序的 DFS 需要先递归访问后续结点，再访问当前结点
    - 类比：前序遍历与后序遍历
  - 因此，需要为每个结点维护三种状态：
    - unvisited：未访问
    - visiting：正在访问，即已经对该结点调用了遍历函数，正在递归遍历后续结点
    - visited：已经完成了对该结点以及递归的遍历
  - 将得到的遍历序列逆序，就是拓扑排序序列，保证对每条边都满足拓扑性质
  - DFS 本身就可以检测回路的存在

伪代码

```
# 输入有向图 G，输出拓扑序列或报告存在回路
def TopSort_DFS(G):
    将所有结点初始化为 unvisited，变量 has_cycle 初始化为 False
    def dfs(u):
        if u 状态为 visiting：表明存在环，has_cycle = True 并返回
        if u 状态为 visited：直接返回
        将 u 的状态设置为 visiting
        对所有邻居 v 分别调用 dfs(v)，如果 has_cycle 改变则直接返回
        将 u 的状态设置为 visited，并添加到 dfs 序列中
    对所有结点 u 依次调用 dfs(u)，如果发现 has_cycle 则报告有环
    将得到的 dfs 序列反转，就得到拓扑序列
```

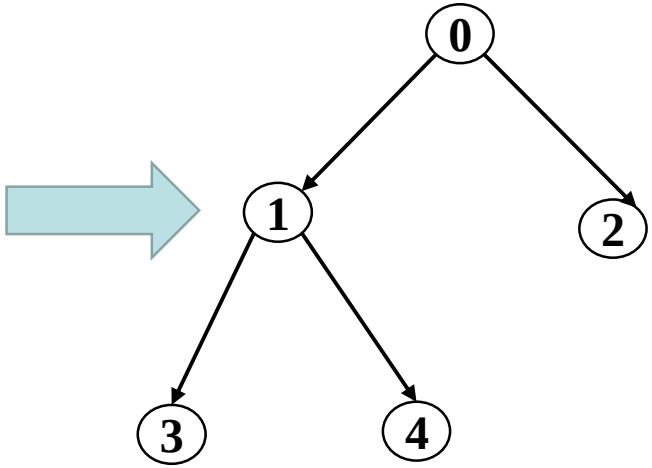
示例

- 实际上可以任意选取顶点开始进行 DFS 遍历
- 拓扑排序的正确性由后序遍历的机制保证



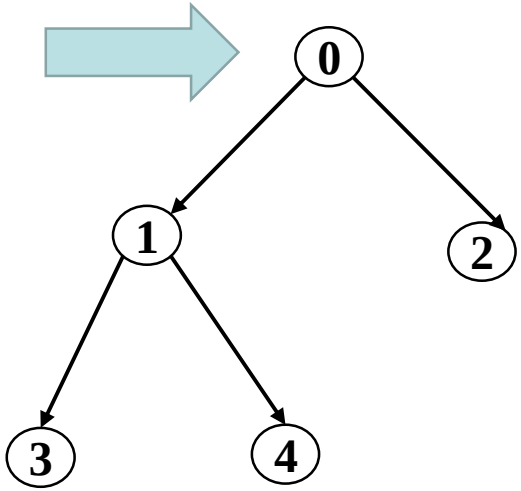
示例

- 实际上可以任意选取顶点开始进行 DFS 遍历
- 拓扑排序的正确性由后序遍历的机制保证
- 由 1 开始 DFS 得到的序列将是 [3, 4, 1]



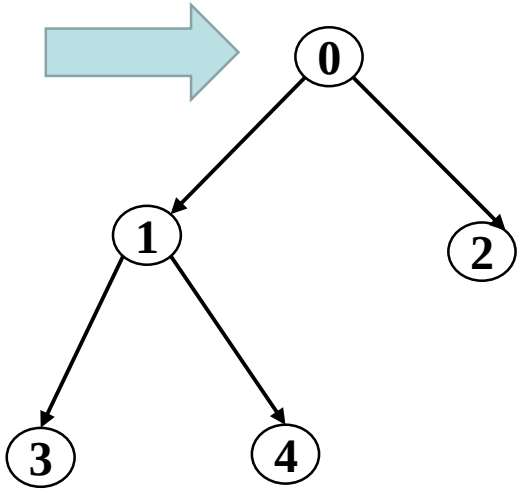
示例

- 实际上可以任意选取顶点开始进行 DFS 遍历
- 拓扑排序的正确性由后序遍历的机制保证
- 由 1 开始 DFS 得到的序列将是 [3, 4, 1]
- 继续由 0 开始 DFS，得到 [2, 0]



示例

- 实际上可以任意选取顶点开始进行 DFS 遍历
- 拓扑排序的正确性由后序遍历的机制保证
- 由 1 开始 DFS 得到的序列将是 [3, 4, 1]
- 继续由 0 开始 DFS，得到 [2, 0]
- DFS 序列：[3, 4, 1, 2, 0]
- 拓扑排序：[0, 2, 1, 4, 3]



判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关

判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关
  - 正确。邻接矩阵的时间空间复杂度都为 $O(n^2)$

判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关
  - 正确。邻接矩阵的时间空间复杂度都为 $O(n^2)$
- 8. 在有向图中，顶点的入度之和等于出度之和；在无向图中，所有顶点的度数之和等于图的边数

判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关
  - 正确。邻接矩阵的时间空间复杂度都为 $O(n^2)$
- 8. 在有向图中，顶点的入度之和等于出度之和；在无向图中，所有顶点的度数之和等于图的边数
  - 错误。无向图中顶点的度数之和等于图的边数的 2 倍

判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关
  - 正确。邻接矩阵的时间空间复杂度都为 $O(n^2)$
- 8. 在有向图中，顶点的入度之和等于出度之和；在无向图中，所有顶点的度数之和等于图的边数
  - 错误。无向图中顶点的度数之和等于图的边数的 2 倍
- 9. 图 G 是一个非连通无向图，共有 28 条边，则该图至少具有 9 个顶点

判断题

- 7. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关
  - 正确。邻接矩阵的时间空间复杂度都为 $O(n^2)$
- 8. 在有向图中，顶点的入度之和等于出度之和；在无向图中，所有顶点的度数之和等于图的边数
  - 错误。无向图中顶点的度数之和等于图的边数的 2 倍
- 9. 图 G 是一个非连通无向图，共有 28 条边，则该图至少具有 9 个顶点
  - 正确。非连通无向图至少具有 2 个连通分量，为了使边数尽可能多，都应该是完全图。具有 7 个顶点的完全图至多具有 7 * 6 / 2 = 21 条边；具有 8 个顶点的完全图至多具有 8 * 7 / 2 = 28 条边

判断题

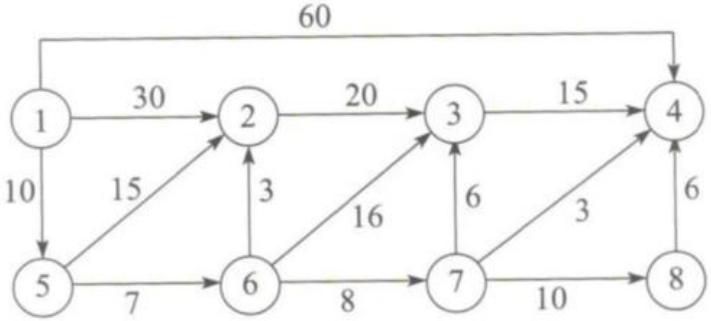
- 10. 从图中的同一个结点出发，图的 BFS 遍历得到的搜索树与 DFS 相比，高度总是较小或相等。

判断题

- 10. 从图中的同一个结点出发，图的 BFS 遍历得到的搜索树与 DFS 相比，高度总是较小或相等。
  - 正确。BFS 中每个结点都按最短路径遍历
  - 若 BFS 搜索树高度为 h，则说明图中存在结点到起始点的最短距离为 h；DFS 遍历过程中该结点在搜索树中的深度不小于 h

简答题

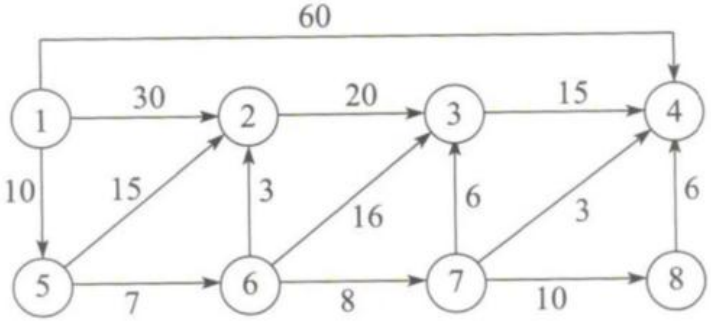
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞

简答题

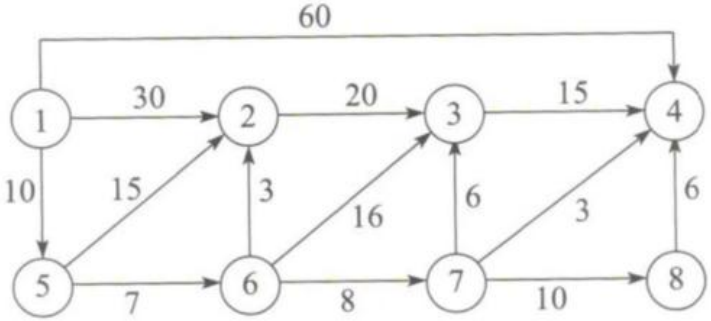
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞

简答题

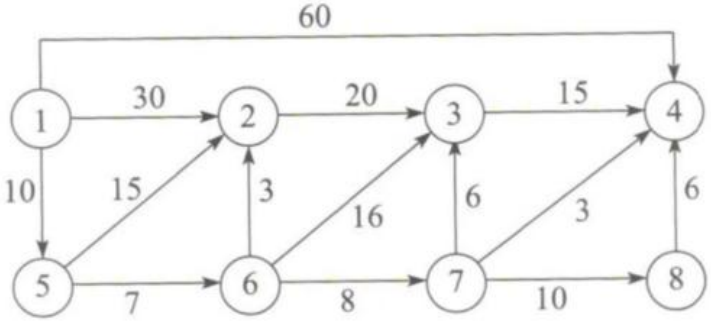
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞

简答题

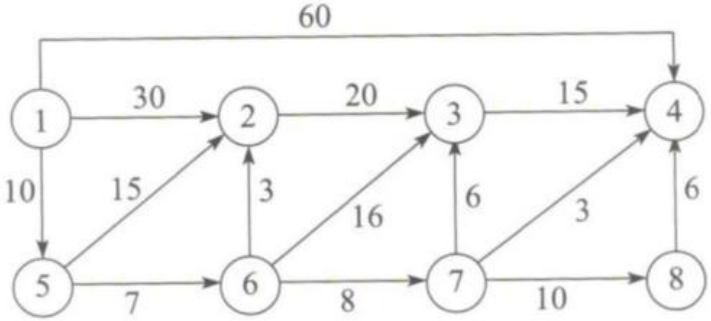
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞
2	{1, 5, 6, 2}	{3, 4, 7, 8}	20	33	60	10	17	25	∞

简答题

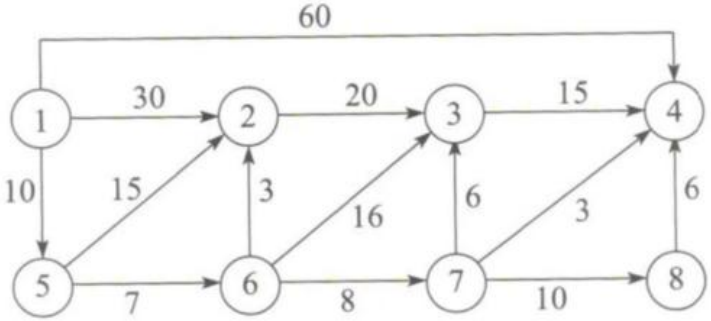
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞
2	{1, 5, 6, 2}	{3, 4, 7, 8}	20	33	60	10	17	25	∞
7	{1, 5, 6, 2, 7}	{3, 4, 8}	20	31	28	10	17	25	35

简答题

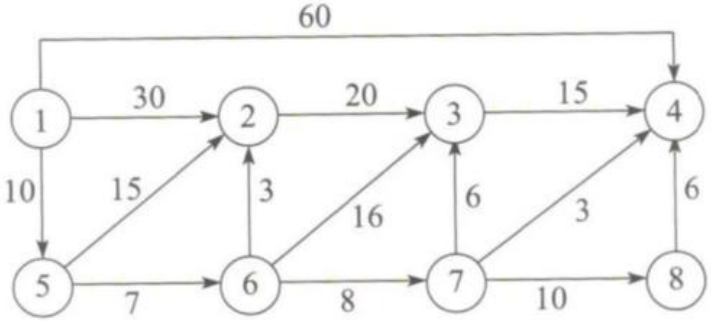
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞
2	{1, 5, 6, 2}	{3, 4, 7, 8}	20	33	60	10	17	25	∞
7	{1, 5, 6, 2, 7}	{3, 4, 8}	20	31	28	10	17	25	35
4	{1, 5, 6, 2, 7, 4}	{3, 8}	20	31	28	10	17	25	35

简答题

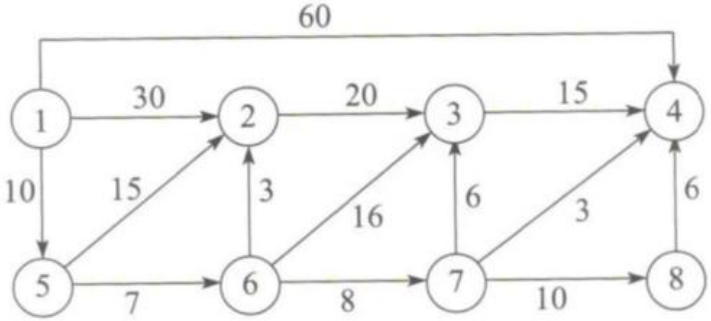
- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞
2	{1, 5, 6, 2}	{3, 4, 7, 8}	20	33	60	10	17	25	∞
7	{1, 5, 6, 2, 7}	{3, 4, 8}	20	31	28	10	17	25	35
4	{1, 5, 6, 2, 7, 4}	{3, 8}	20	31	28	10	17	25	35
3	{1, 5, 6, 2, 7, 4, 3}	{8}	20	31	28	10	17	25	35

简答题

- 运行 Dijkstra 算法求出顶点 1 到各顶点的最短路径，将算法运行各步的状态填入下表



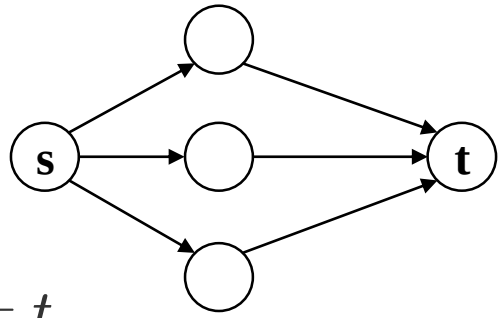
所选顶点	U(已确定最短路径的集合)	T(未确定最短路径的集合)	2	3	4	5	6	7	8
初态	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞
5	{1, 5}	{2, 3, 4, 6, 7, 8}	25	∞	60	10	17	∞	∞
6	{1, 5, 6}	{2, 3, 4, 7, 8}	20	33	60	10	17	25	∞
2	{1, 5, 6, 2}	{3, 4, 7, 8}	20	33	60	10	17	25	∞
7	{1, 5, 6, 2, 7}	{3, 4, 8}	20	31	28	10	17	25	35
4	{1, 5, 6, 2, 7, 4}	{3, 8}	20	31	28	10	17	25	35
3	{1, 5, 6, 2, 7, 4, 3}	{8}	20	31	28	10	17	25	35
8	{1, 5, 6, 2, 7, 4, 3, 8}	{}	20	31	28	10	17	25	35

简答题

- 请给出一个线性时间的算法（即时间复杂度为 O(n+e)），算法的输入为一个有向无环图 G=(V, E) 以及两个结点 s, t，算法的输出是从结点 s 到 t 的简单路径的数量。
- 仅需要计算出简单路径的数量，不要求将简单路径本身列举出来。
- 简单路径：除了起点与终点可以相同，路径中的任意两个结点不相同
  - 也有其它定义，要求简单路径中任意两个结点都不相同；并区分出来简单回路

简答题

- 可以使用动态规划的思想来解决本题
  - 原问题：s 到 t 的路径数量
  - 先行动一步：s 到邻居 u，再由 u 到 t
  - 用 F(s, t) 表示原问题的解，则有



$$F(s,t) = \begin{cases} 1, & \text{if } s = t \\ \sum_{u \in Adj[s]} F(u,t) & \text{if } s \neq t \end{cases}$$

  - 在求解所有子问题之前，我们需要按照子问题之间的依赖关系，决定求解的顺序

简答题

$$F(s,t) = \begin{cases} 1, & \text{if } s = t \\ \sum_{u \in Adj[s]} F(u,t) & \text{if } s \neq t \end{cases}$$

- 子问题的依赖关系：
  - 子问题 F(s, t) 依赖于所有的 F(u, t)，u 为 s 的邻居
  - 即对于每一条有向边 <u, v>，必须先求解 F(v, t)，后求解 F(u, t)
  - 实际上，这就是图 G 的拓扑排序
- 解答算法：
  - 对 G 进行拓扑排序，并对拓扑排序序列取逆序得到序列 P
  - 按照序列 P 的顺序，求解所有子问题
  - 最终返回 F(s, t) 即可
  - 时间复杂度为 O(n+e)

算法填空

- 完成下列算法，计算无向图中所有连通分量的结点个数。
  - 输入：图的结点数，边数和边的列表
  - 输出：每个连通分量的结点个数

输入样例：

5

[(0, 1), (1, 2), (3, 4)]

输出样例：

[3, 2]

算法填空

```
def dfs(node, visited, graph, n):
    count = 1
    visited[node] = True
    for i in range(n):
        if i in graph[node] and ______(1)______:
            count += _________(2)_________
    return count

def count_components(n, edges):
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        _________(3)_________
        _________(4)_________

    visited = [False] * n
    components = []
    for i in range(n):
        if __________(5)___________:
            components.append(____________(6)_____________)
    return components

n = int(input())
edges = eval(input())
print(count_components(n, edges))
```

算法填空

```
def dfs(node, visited, graph, n):
    count = 1
    visited[node] = True
    for i in range(n):
        if i in graph[node] and not visited[i]:
            count += dfs(i, visited, graph, n)
    return count

def count_components(n, edges):
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    components = []
    for i in range(n):
        if not visited[i]:
            components.append(dfs(i, visited, graph, n))
    return components

n = int(input())
edges = eval(input())
print(count_components(n, edges))
```

算法填空

- 堆排序：输入若干个整数，下面的程序使用堆排序算法对这些整数从小到大排序，请填空。程序中建立的堆是大顶堆，最大元素在堆顶。
- 输入样例：1, 3, 43, 8, 7
- 输出样例：1, 3, 7, 8, 43

算法填空

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[i] < arr[left]:
        largest = left
    if right < n and arr[largest] < arr[right]:
        largest = right
    if largest != i:
        _____________(1)_____________
        _____________(2)_____________
def buildMaxHeap(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        ___________(3)___________
def heapSort(arr):
    n = len(arr)
    buildMaxHeap(arr)
    for i in range(n - 1, 0, -1):
        _____________(4)_____________
        _____________(5)_____________
    return arr

a = list(map(int, input().split()))
heapSort(a)
for x in a:
    print(x, end=" ")
```

算法填空

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[i] < arr[left]:
        largest = left
    if right < n and arr[largest] < arr[right]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def buildMaxHeap(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
def heapSort(arr):
    n = len(arr)
    buildMaxHeap(arr)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

a = list(map(int, input().split()))
heapSort(a)
for x in a:
    print(x, end=" ")
```

上机作业：拨钟问题

- 题目：有 9 个时钟，排成 3*3 的矩阵；以及 9 种不同的操作 1~9，每种操作都可以使不同范围内的时钟前进一步。求最短的移动序列，使所有时钟指向 12 点

```
|-------|    |-------|    |-------|
|       |    |       |    |   |   |
|---O   |    |---O   |    |   O   |
|       |    |       |    |       |
|-------|    |-------|    |-------|
    A            B            C

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   O   |    |   O   |    |   O   |
|   |   |    |   |   |    |   |   |
|-------|    |-------|    |-------|
    D            E            F

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   O   |    |   O---|    |   O   |
|   |   |    |       |    |   |   |
|-------|    |-------|    |-------|
    G            H            I
```

移动	影响的时钟
1	ABDE
2	ABC
3	BCEF
4	ADG
5	BDEFH
6	CFI
7	DEGH
8	GHI
9	EFHI

上机作业：拨钟问题

- BFS 解法：
  - 该问题很容易建模成有向图，9 个时钟的状态组成图中的结点
  - 每个结点具有 9 条出边，对应 9 种操作
  - 题目要求求出最短路径，因此优先考虑使用 BFS
- 但是，基本的遍历算法可能导致 TLE
  - 总的状态数与边数数量过大
  - 如果使用 DFS，遍历所有状态，比较最优的可行解并找到最优解，与 BFS 解法在最坏情况下的复杂度相同
  - 实际耗时也取决于实现，即复杂度中省略的常数项
  - 因此部分的 BFS / DFS 实现也 accepted

上机作业：拨钟问题

- 进行可行性剪枝：
  - 对于单个时钟，只有有限个操作能够影响
  - 例如，对于时钟 A，仅有操作 1, 2, 4
  - 如果确定了 1, 2 操作的次数，则可以直接限制操作 4 的操作次数
  - 假定最短操作序列中，较小编号的操作都在较大的之前
  - 这样，就可以确定后续操作的次数，或者缩小搜索范围

```
|-------|    |-------|    |-------|
|       |    |       |    |   |   |
|---o   |    |---o   |    |   o   |
|       |    |       |    |       |
|-------|    |-------|    |-------|
    A            B            C

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   o   |    |   o   |    |   o   |
|   |   |    |   |   |    |   |   |
|-------|    |-------|    |-------|
    D            E            F

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   o   |    |   o---|    |   o   |
|   |   |    |       |    |   |   |
|-------|    |-------|    |-------|
    G            H            I
```

移动	影响的时钟
1	ABDE
2	ABC
3	BCEF
4	ADG
5	BDEFH
6	CFI
7	DEGH
8	GHI
9	EFHI

上机作业：拨钟问题

- 基于 BFS 进行可行性剪枝：
  - 具体来说，操作 1, 2, 3 无法进行剪枝，正常遍历。在此基础上：
    - 4 → A，5 → B，6 → C，7 → D，8 → G，9 → I
- 也可以直接使用 DFS，或者简单遍历，结合剪枝方法实现

```
|-------|    |-------|    |-------|
|       |    |       |    |   |   |
|---O   |    |---O   |    |   O   |
|       |    |       |    |       |
|-------|    |-------|    |-------|
    A            B            C

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   O   |    |   O   |    |   O   |
|   |   |    |   |   |    |   |   |
|-------|    |-------|    |-------|
    D            E            F

|-------|    |-------|    |-------|
|       |    |       |    |       |
|   O   |    |   O---|    |   O   |
|   |   |    |       |    |   |   |
|-------|    |-------|    |-------|
    G            H            I
```

移动	影响的时钟
1	ABDE
2	ABC
3	BCEF
4	ADG
5	BDEFH
6	CFI
7	DEGH
8	GHI
9	EFHI

上机作业：Highways

- 题目：Flotopia 国家中有 N 个城镇，城镇间存在 M 条高速公路，但目前没能连通全部城镇。给出所有城镇的二维坐标，将直线距离视为公路长度。找出使全部城镇连通，且总花费最小的建造方案。

上机作业：Highways

- 题目：Flotopia 国家中有 N 个城镇，城镇间存在 M 条高速公路，但目前没能连通全部城镇。给出所有城镇的二维坐标，将直线距离视为公路长度。找出使全部城镇连通，且总花费最小的建造方案。
  - 该问题显然与最小生成树有关
  - 在已有边集的基础上，将其扩充为一个最小生成树？
    - 已有的边集并不一定能构成最小生成树
    - 例如，已有的高速公路完全可能包含环
    - 建造方案可能只利用了已有公路的一部分

上机作业：Highways

- 解法一：基于 Kruskal 算法
  - 为所有可能的点对之间计算距离，排序；
  - 根据已有的公路合并连通分量
    - 此时，已经建造的公路对应的边，将位于连通分量内部
  - 在此基础上，继续运行 Kruskal 算法，保存所有接下来添加的边
  - 注意：Kruskal 算法的退出条件，即连通分量数 > 1
  - 在处理已有的公路时，要正确维护连通分量数
  - 不能简单地将连通分量数减去 M

上机作业：Highways

- 解法二：基于 Prim 算法
  - 考虑从零开始构建最小生成树
  - 区别在于，已经存在的公路不需要再重复建造，即这种公路的成本为 0
  - 仍然计算所有点对之间的距离，然后将已存在的公路对应的边的权重设置为 0
  - 在此基础上，运行 Prim 算法，得到最小生成树
  - 从中删除权重为 0 的边，就得到原问题的解

上机作业：Arbitrage

- 题目：给定 N 种货币，以及 M 个汇率，表示一单位 A 货币可以交换得到的 B 货币数量。换汇是单向的。要求判断，利用给定的操作能否实现套利
  - 首先，可以用有向图来建模该问题
  - 套利是指，从一种货币出发，经一系列有向边回到自身，但汇率乘积大于 1，即检测乘积大于 1 的环
  - 可以联想到 Floyd 算法
  - Floyd 算法计算最短路，同时可以用于检测图中的负权环

上机作业：Arbitrage

- 基于 Floyd 算法的解答
  - 可以将每条边的权重设置为 – log(W)，W 为原始汇率
  - 汇率之积大于 1，就等价于权重之和小于 0
  - 自身到自身的权重为 0，不存在的边权重为 INF
- 也可以改进 Floyd 算法来计算该问题
  - 计算两条路径的总权重时，应该由相加改为相乘
  - Floyd 算法求最短路，本问题中应该求最长路
  - 因为目标是找到可以套利的回路，希望使汇率乘积尽量大
  - 自身到自身的权重为 1，不存在的边权重为 0

上机作业：Arbitrage

- Corner Case：
  - 有向图中可能存在自环边
  - 即可能存在由一种货币指向自身，且汇率大于 1 的情况
  - 上述算法可以正确处理这种情况，但是需要注意实现上的问题
  - 初始化距离矩阵，以及读取输入的顺序不能颠倒
  - 初始化时，将 Dist[i][i] 设置为 1，而读取输入时可能将 Dist[i][i] 设置为其他值，即会相互覆盖

图相关算法总结

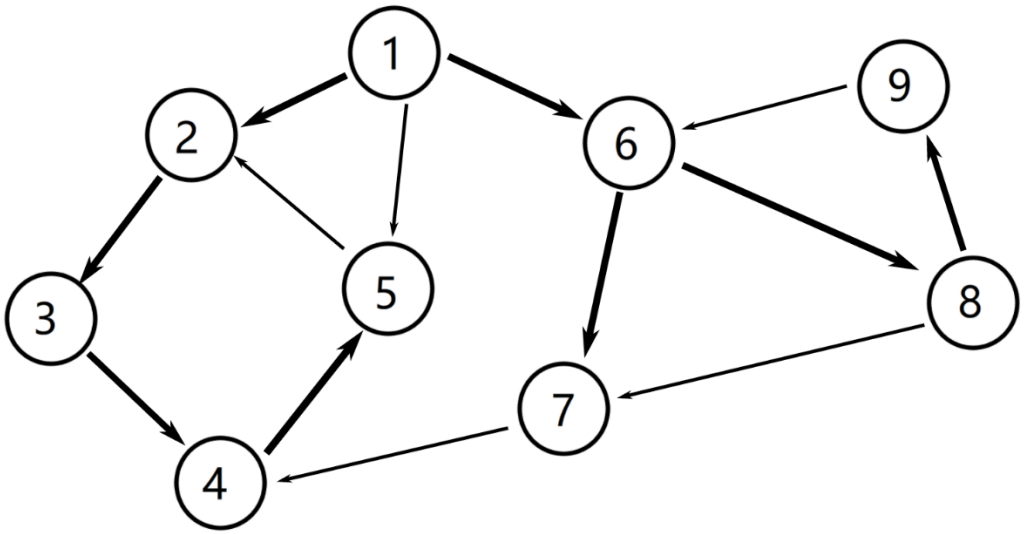
- 搜索与遍历算法：DFS、BFS 算法
  - 实际搜索问题建模、连通性与连通分量判定、回路检测
- 最小生成树算法：Prim、Kruskal 算法
  - 最小生成树，以及其变种问题
- 最短路算法：Dijkstra、Floyd 算法
  - 最短路，以及其变种问题
  - 负权环的检测
  - 有向图连通性的判定
- 拓扑排序算法
  - 回路检测
  - * 关键路径问题

补充内容：强连通分量

- 利用已经学过的算法，如何求解一个无向图的连通分量？
  - DFS / BFS 每一次搜索都遍历了一个连通分量
  - 无向图中结点的连通是等价关系，使用 UnionFind
- 思考：如何求解一个有向图的所有强连通分量？

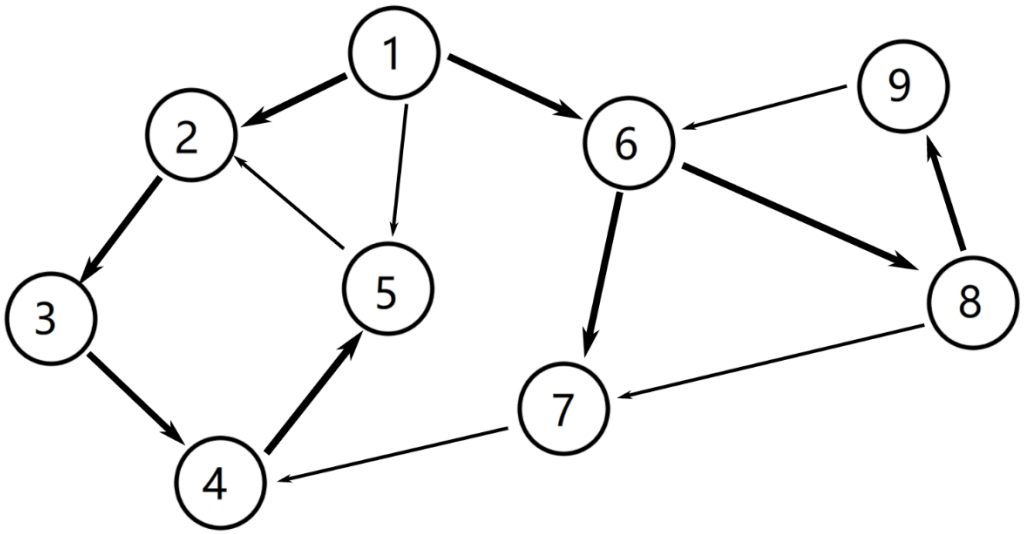
补充内容：强连通分量

- 思考：如何求解一个有向图的所有强连通分量？



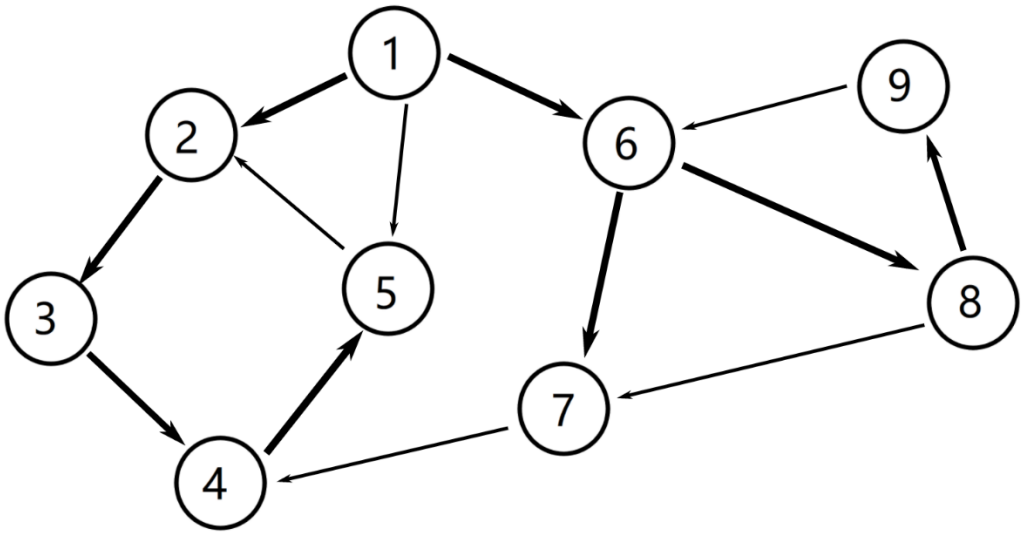
补充内容：强连通分量

- 思考：如何求解一个有向图的所有强连通分量？
  - 观察：寻找有向边组成的回路
  - 回路应该包含尽可能多的点（即强连通分量的极大性）



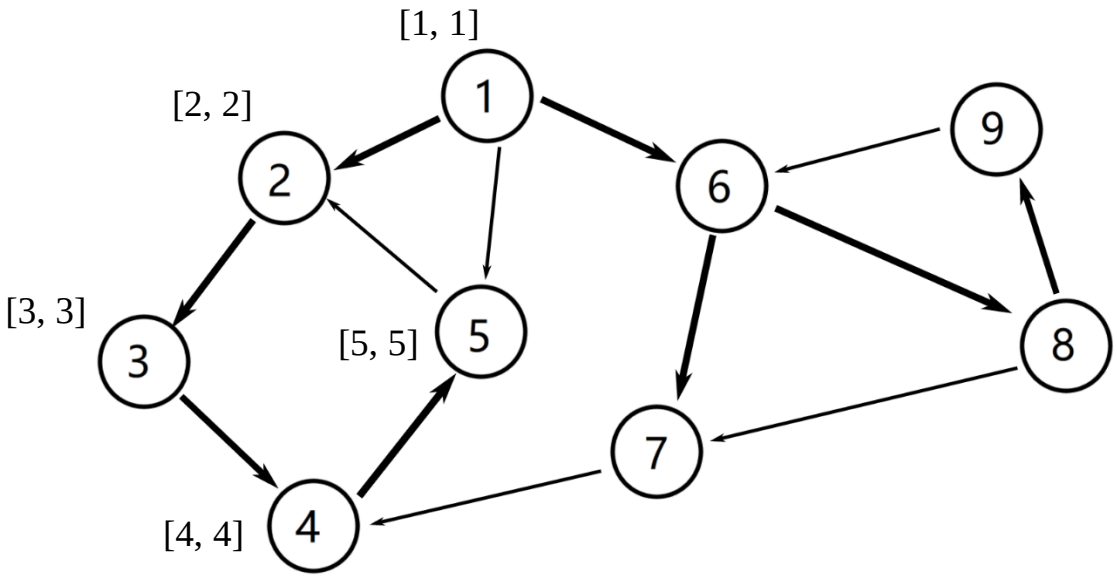
Tarjan 算法

- Tarjan 算法基于 DFS，可以用于求解有向图的强连通分量
  - Tarjan 算法为每个结点维护两个值
  - pre[i] 表示该结点在 DFS 中被遍历的次序，父结点 pre < 子结点 pre
  - low[i] 表示该结点可达的搜索树中的祖先，具有的最小 pre 值



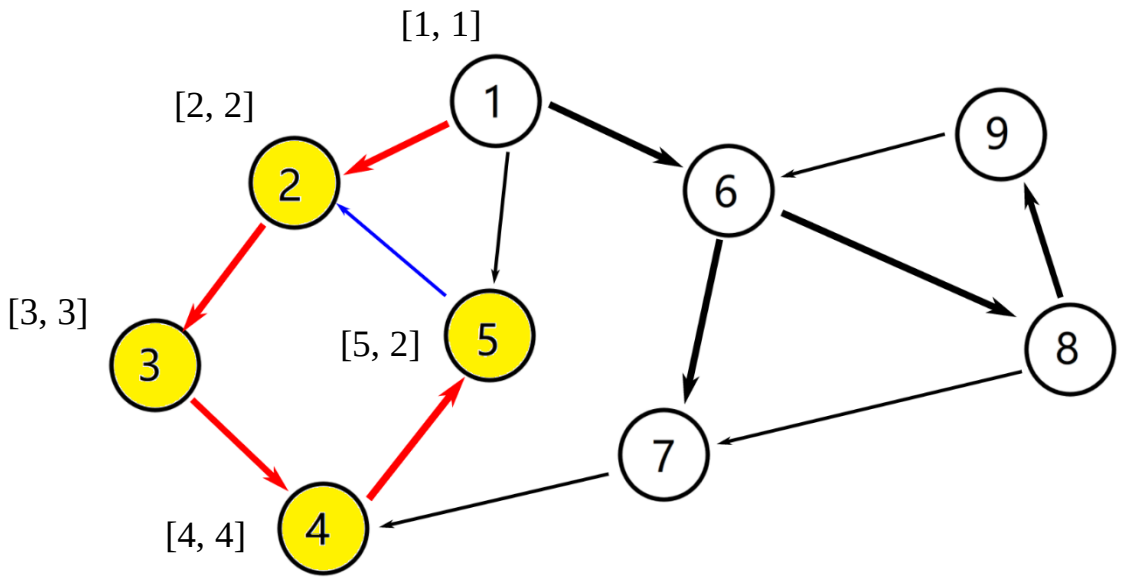
Tarjan 算法

- 进行 DFS 时，为新访问的结点更新 pre 与 low 值；
- 同时记录正在遍历的结点 (on_stack)



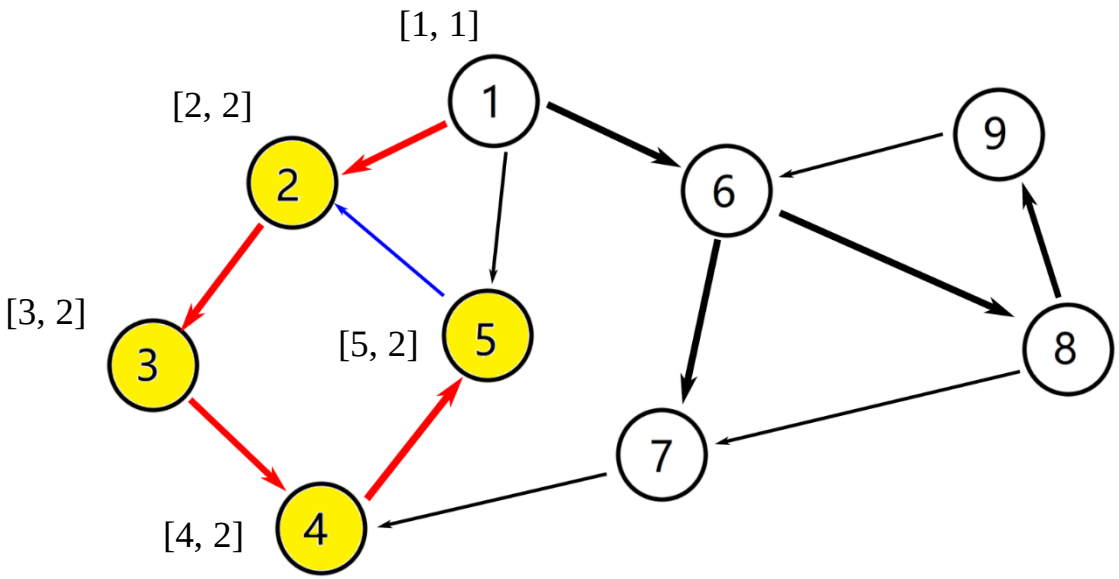
Tarjan 算法

- 进行 DFS 时，为新访问的结点更新 pre 与 low 值；
- 同时记录正在遍历的结点 (on_stack)
- 此时，由于结点 2 on stack，结点 5 将其 low 值更新为 2
- 到达搜索树的某个分支底部时，没有可以继续前进的路径，回溯



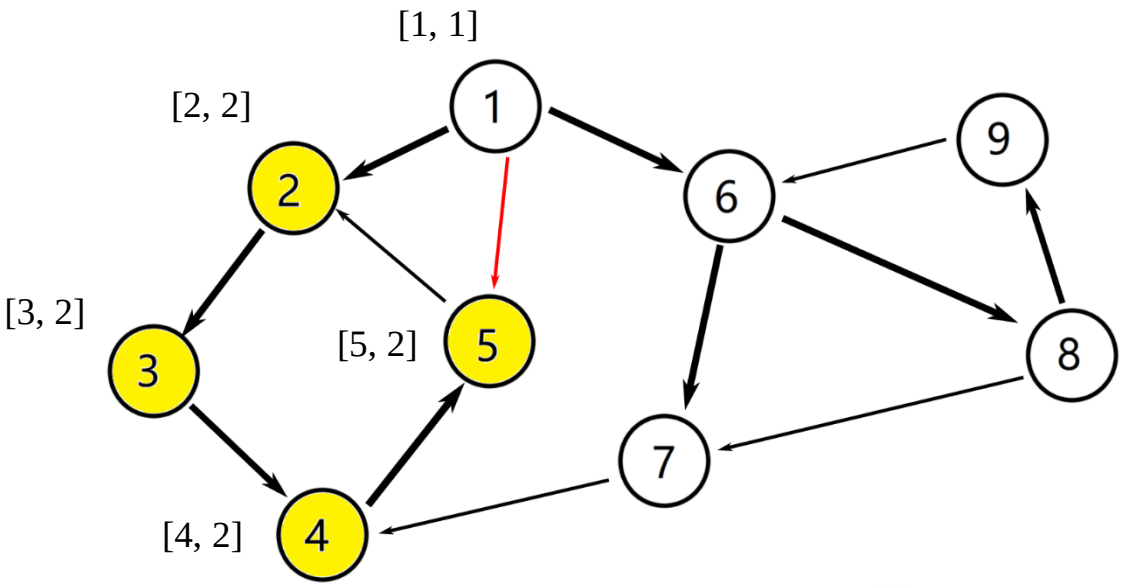
Tarjan 算法

- 回溯过程中，父结点依据子结点的 low 值更新自身的 low 值
- 自身的 pre 值与更新后的 low 值相等时，就找到了一个强连通分量，更新 on_stack 状态，即将强连通分量中的点出栈



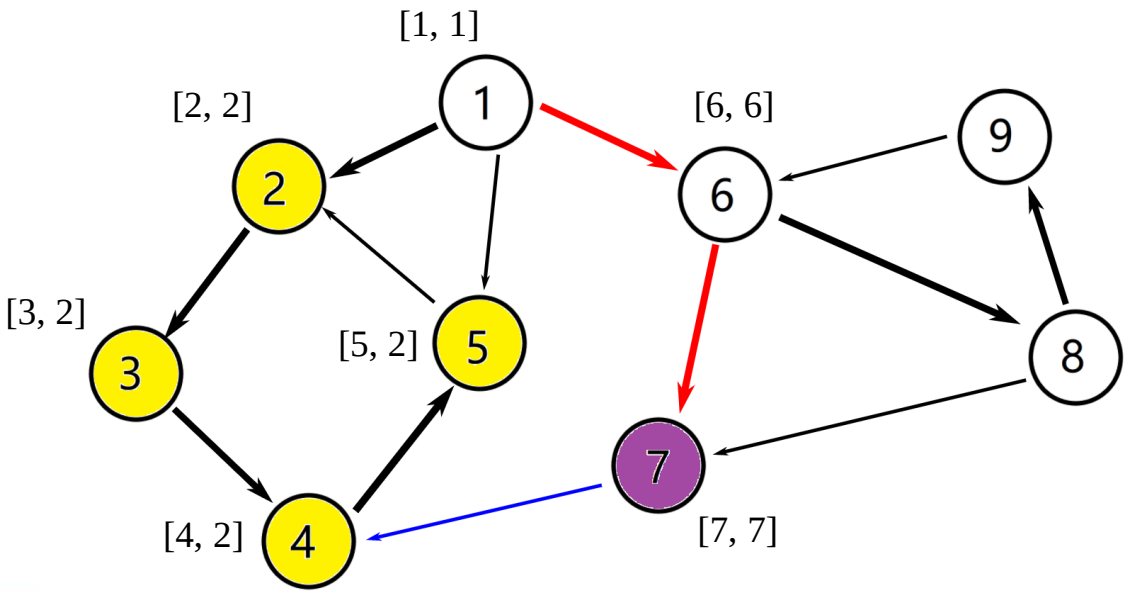
Tarjan 算法

- 回溯至 1 后继续遍历
- 由于 5 已经访问，不会再递归访问 5，但会进行更新 low 值的判断



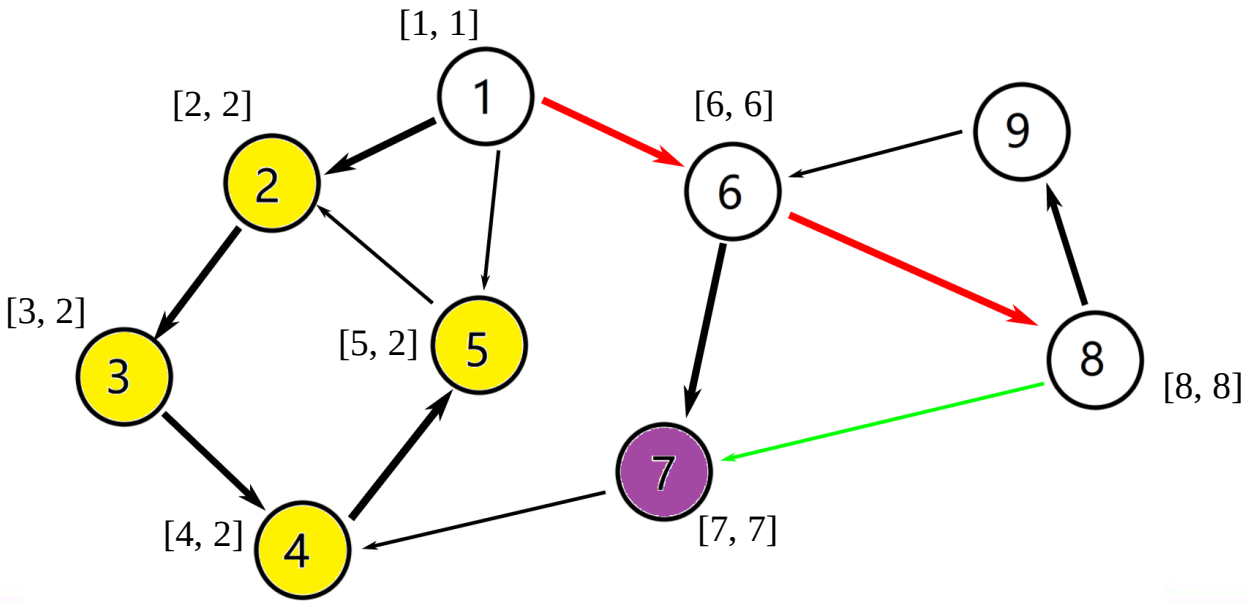
Tarjan 算法

- 遍历至 7 时，由于 4 已经访问，7 已经到达了搜索树的底部
- 同时，由于 4 不在栈中，low[7] 无法更新，已经达成了 pre == low
- 即发现了一个连通分量，结点包括 7，以及栈中压在 7 上面的元素（实际上只有 7）



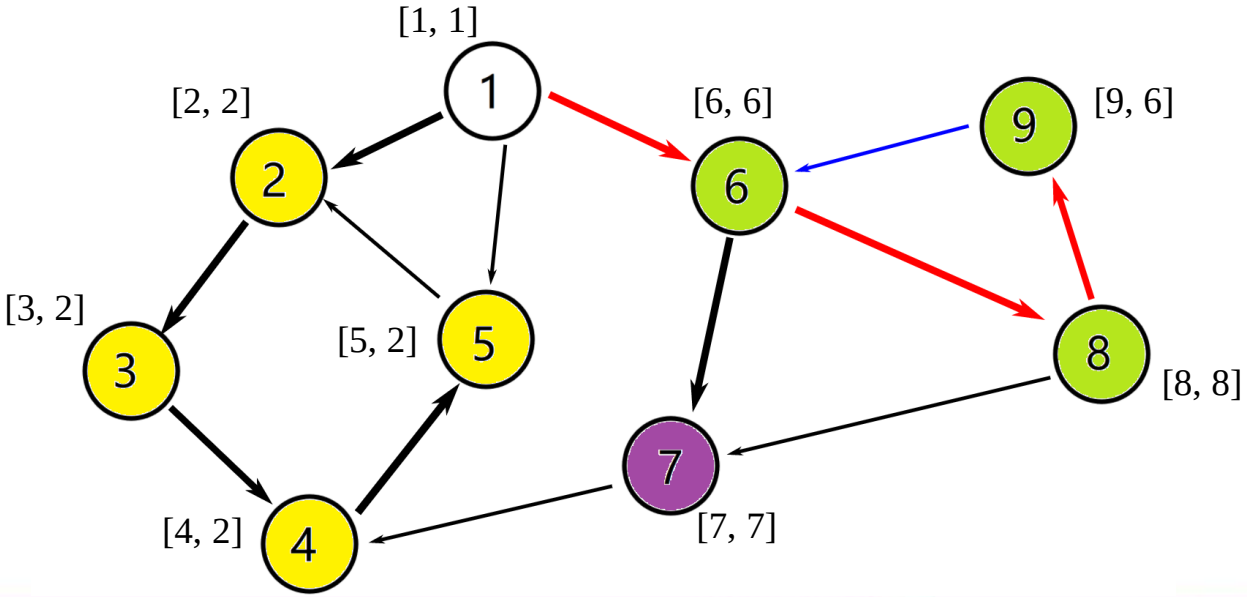
Tarjan 算法

- 回溯至 6 继续遍历
- 遍历至 8 时，对于结点 7，由于其已经遍历过，且不在栈中，不会造成影响



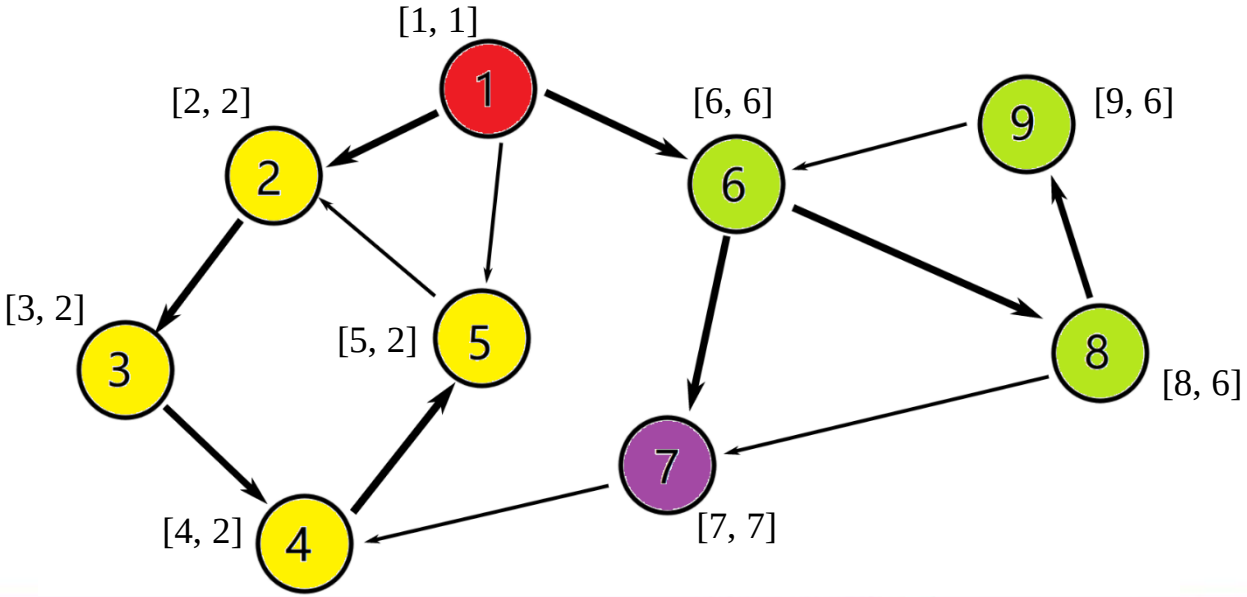
Tarjan 算法

- 回溯至 6 继续遍历
- 遍历至 8 时，对于结点 7，由于其已经遍历过，且不在栈中，不会造成影响
- 遍历至 9 时，再次到达了搜索树的底部，且可以更新 low 值
- 回溯至结点 6，发现了一个连通分量



Tarjan 算法

- 最后回溯至结点 1，栈中仅有结点 1，无法更新 low 值
- 结点 1 自成一个强连通分量



Tarjan 算法：伪代码

```
# 输入有向图 G，输出强连通分量 (Strongly Connected Components, SCC)
def Tarjan(G):
    初始化 pre, low, on_stack 列表，时钟 clock，栈 s，以及 SCC 结果列表
    def dfs(u):
        pre[u], low[u] 赋值为 clock，更新clock，将 u 入栈，更新on_stack[u]
        对 u 的所有邻居 v:
            如果 v 未访问，dfs(v)，low[u]=min(low[u], low[v])
            否则，若 v 在栈中，low[u]=min(low[u], pre[v])
        如果 low[u]==pre[u]，弹出 s 中 u 以及之上的元素，组成 SCC
    对 G 中所有结点 u:
        如果 u 未访问，dfs(u)
```

Bellman-Ford 算法

- 回顾 Dijkstra 算法，进行了一系列的“更新”操作：
  - 结点 v 的原距离值估计为 Dist[v]
  - 考虑结点 u 以及 <u, v>，可以更新为 min(Dist[v], Dist[u]+W(u, v))
- 该操作具有如下的性质
  - 假设 $p = v_0, v_1, \cdots, v_n$ 是 v_0 到 v_n 的最短路径
  - 我们按次序，依次对 $< v_0, v_1 >, \cdots, < v_{n-1}, v_n >$ 进行更新操作；此时 Dist[v_n] 一定等于最短路径长度，且与其他更新操作无关。
  - 使用归纳法易证

Bellman-Ford 算法

- Bellman-Ford 算法的思想：
  - 首先，最短路上不可能包含回路，即每个结点最多出现一次；
  - 这意味着最短路的路径数量最多为 n – 1
  - 前文叙述的性质，要求我们必须按次序对最短路径上的边进行更新操作，完成更新后，就求得了最短路
  - 因此，我们可以对图中全部的边进行更新操作，共进行 n – 1 轮
  - 对任何最短路径，n – 1 轮更新后，所有的边一定都被按次序更新了

Bellman-Ford 算法

- Dijkstra 算法：
  - 每轮更新一个结点的所有出边，共 n – 1 轮
  - 每条边都只被更新一次
  - 每轮更新后，按贪心法选择一个点确认最短路径
  - 时间复杂度：$O(n^2)$ 或 $O((n+e)\log n)$
- Bellman-Ford 算法：
  - 每轮更新所有边，共 n – 1 轮
  - 每条边被更新 n – 1 次
  - 每轮更新后无需做任何事情，n – 1 轮后自动得到所有点的最短路径
  - 时间复杂度：$O(n*e)$

Bellman-Ford 算法：伪代码

```
# 输入有向图 G，源点 s，输出单源最短路径长度
def Bellman-Ford(G, s):
    初始化距离列表 Dist
    for k = 0 to n – 1:
        对图中的每个结点 v_i:
            对于结点 v_i 的所有出边 < v_i, v_j >:
                Dist[j] = min(Dist[j], Dist[i] + W(i, j))
    返回 Dist 值
```

- 如果需要记录最短路本身，另外维护前驱列表 Prev 即可
- Bellman-Ford 算法还可以检测由源点可达的负环，只要在更新结束后，再进行一轮额外的更新。如果有路径长度还可以变小，就意味着存在源点可达的负环。

Johnson 算法

- 对于多源最短路问题，Floyd 算法的复杂度为 $O(n^3)$
- 重复调用 Dijkstra 算法，复杂度为 $O(n^2\log n + n*e\log n)$
- 然而，前者可以很好地处理负边和负环
- Johnson 算法是基于 Dijkstra 算法的改进，重复调用 Dijkstra 算法计算多源最短路，并具有处理负边和负环的能力
- 思考：如果图 G 中含有负权边，我们通过给图中的每条边的权值都加上一个常数 c，消除负权边，再运行 Dijkstra 算法，能够正确计算最短路吗？

Johnson 算法

- 对于多源最短路问题，Floyd 算法的复杂度为 $O(n^3)$
- 重复调用 Dijkstra 算法，复杂度为 $O(n^2\log n + n*e\log n)$
- 然而，前者可以很好地处理负边和负环
- Johnson 算法是基于 Dijkstra 算法的改进，重复调用 Dijkstra 算法计算多源最短路，并具有处理负边和负环的能力
- 思考：如果图 G 中含有负权边，我们通过给图中的每条边的权值都加上一个常数 c，消除负权边，再运行 Dijkstra 算法，能够正确计算最短路吗？
  - 不可以。最短路并不一定具有最少的边数

Johnson 算法

- Johnson 算法的思想为重新赋予权值：
  - 如果图 G 中包含负边，但不包含负环，我们可以计算出一组新的权值，得到图 G’，并基于这组权值运行 Dijkstra 算法
  - 图 G’ 需要满足下面两条性质
    - 1. p 在 G 中是一条最短路径，当且仅当 p 在 G’ 中是最短路径
    - 2. 对于所有的边，权值为非负值

Johnson 算法

- 引理：我们可以将每个结点 v 映射为一个实数 h(v)，并基于此定义新的权重：

$$W'(u,v) = W(u,v) + h(u) - h(v)$$

- 则原权重 W 下的路径 p 为最短路径，当且仅当其在权重 W' 下也是最短路径。
  - 可以将 h(v) 想象为结点 v 的“高度”
  - 这样，u 到 v 的最短路径长度就增加了由 u 到 v 所变化的高度
  - 无论中间经历了哪些边，都不会改变起点与终点之间的高度差
  - 即，假设 p 是 v_0 到 v_n 的任意一条路径，有

$$W'(p) = W(p) + h(v_0) - h(v_n)$$

- 同样，原图中包含负环也等价于修改后的图中包含负环

Johnson 算法

- 这样，我们就只需要找到一个函数 h，使得所有边权非负
  - 我们引入一个新结点 s，并向原图的所有结点连边，且权重为 0
  - h(u) 定义为 s 到 u 的最短路径长度
  - 我们知道，对于边 <u, v>，成立 $h(v) \leq h(u) + W(u,v)$
  - 因此 $W'(u,v) = W(u,v) + h(u) - h(v) \geq 0$
- 计算 h(u) 的过程需要调用 Bellman-Ford 算法
- 同时，还可以利用 Bellman-Ford 算法检测 s 可达的负环，实际上就是 G 中可能存在的任意负环
- 依照上述方法构建新图，将每个结点依次作为源点，调用 Dijkstra 算法，计算最短路

Johnson 算法：伪代码

```
# 输入有向图 G，输出所有结点对间的最短路径
def Johnson(G):
    加入新结点 s 并连边，构建图 G’
    调用 Bellman-Ford(G’, s)，
    若图中存在负环，算法直接返回
    依据源点 s 的最短路结果，更新 G’ 中的边权
    对 G’ 中除 s 外的所有结点 u:
        调用 Dijkstra(G’, u)，得到 G’ 下的最短路径
        计算原图 G 下的最短路径长度
    返回最短路径长度矩阵
```

Johnson 算法

- Johnson 算法的时间复杂度分析
  - 该算法使用了 Bellman-Ford 算法与 Dijkstra 算法作为子程序
  - 首先在 G’ 上运行 Bellman-Ford 算法，复杂度为 O(n*e)
  - 然后调用 n 次 Dijkstra 算法
  - 由 G’ 到 G 的转换代价为 O(n)
  - 总的时间复杂度为 $O(n^2 \log n + n*e \log n)$

上机模拟考试

- 时间：本周四上机课 15:10 – 17:00
- 地点：理科一号楼三楼 7，8 号机房
- 考试重点：
  - 数据结构的基础操作
  - 基础算法设计：递归、贪心、动态规划
  - 栈与队列的应用
  - 二叉树的基本操作、二叉堆、并查集
  - 图的搜索与遍历、连通性、最短路算法、最小生成树算法、拓扑排序
  - 排序与检索算法

上机模拟考试

- 考试提示：
  - 重点是所学知识的灵活运用，而不在于数据结构本身的实现细节
  - 共包括6道题目，总体上由易到难排列；与作业题目难度相当
  - 题目按梯度给分；注意考试策略与时间安排
  - 注意算法的时间复杂度，仅通过部分测试点也无法获得分数